

C++ STL Algorithms

The Power of Standard Functions

A 10-Page Technical Reference Guide

Optimization | Predicates | Binary Search | Modern C++

1. Introduction to STL Algorithms

In C++, **Algorithms** are a collection of highly optimized functions designed to operate on a range of elements. They are the "verbs" of the STL, while containers are the "nouns."

Key Concept: Algorithms do not work on containers directly. They work through **Iterators**. This separation allows one `std::sort` function to work on a `vector`, a `deque`, or even a plain C-style array.

To use them, you must include the `<algorithm>` header. For mathematical algorithms like sums, include `<numeric>`.

2. Foundational Algorithms: Sort & Reverse

These are the most used functions in competitive programming and day-to-day development.

std::sort()

Sorts elements in a range in ascending order. Time Complexity: $O(N \log N)$.

```
vector<int> v = {5, 2, 8, 1};  
sort(v.begin(), v.end()); // {1, 2, 5, 8}
```

std::reverse()

Flips the order of elements in constant space.

```
reverse(v.begin(), v.end()); // {8, 5, 2, 1}
```

Note: You cannot use `std::sort` on a `std::list`. Lists have their own member function `list.sort()` because they aren't random access.

3. Non-Modifying: Find & Count

These algorithms allow you to inspect data without changing it.

std::find()

Searches for a specific value and returns an iterator to it. If it fails, it returns the end () iterator.

```
auto it = find(v.begin(), v.end(), 5);  
if (it != v.end()) cout << "Found at index " << distance(v.begin(), it);
```

std::count()

Returns the number of times a value appears in a range.

```
int tally = count(v.begin(), v.end(), 10); // How many 10s?
```

4. Max, Min, and Sums

Quickly identifying boundaries and totals in your data.

Min and Max Element

These functions return an **iterator** to the smallest or largest item.

```
auto max_it = max_element(v.begin(), v.end());  
cout << "Highest value is: " << *max_it;
```

std::accumulate()

Found in <numeric>. Calculates the sum of all elements in a range starting from an initial value.

```
#include <numeric>  
int total = accumulate(v.begin(), v.end(), 0);
```

5. Binary Search (The $O(\log N)$ Speed)

Binary search is significantly faster than `find()`, but it has one strict requirement: **The container must be sorted.**

`binary_search()`

Simply returns true or false.

```
bool exists = binary_search(v.begin(), v.end(), 42);
```

Lower and Upper Bound

- **lower_bound:** Iterator to first element $\geq$ value.
- **upper_bound:** Iterator to first element $>$ value.

```
auto it = lower_bound(v.begin(), v.end(), 10);
```

6. Modern C++: All, Any, and None

Introduced in C++11, these allow you to check properties of all elements using **Lambda Functions**.

```
// Check if ALL numbers are even
bool allEven = all_of(v.begin(), v.end(), [](int x) {
    return x % 2 == 0;
});

// Check if ANY number is negative
bool hasNeg = any_of(v.begin(), v.end(), [](int x) {
    return x < 0;
});
```

These functions return a boolean immediately after the condition is satisfied (short-circuiting).

7. The Erase-Remove Idiom

A common mistake is thinking `std::remove` actually deletes elements. It doesn't! It just moves the items to be deleted to the end of the container and returns a new end.

The Process: 1. `remove()` shifts elements. 2. `erase()` actually shrinks the container size.

```
// Remove all 10s from a vector
v.erase(remove(v.begin(), v.end(), 10), v.end());
```

This "idiom" is a must-know for interviews because it shows you understand how STL memory management works.

8. Map & Set Intrinsic Algorithms

While STL algorithms are general, binary search-based containers like `std::set` and `std::map` have their own **member functions** for searching.

Scenario	Use Global Algorithm	Use Member Function
Searching a Vector	<code>std::find(v.begin(), v.end(), x)</code>	N/A
Searching a Set	Don't! (Slow $O(N)$)	<code>mySet.find(x)</code> (Fast $O(\log N)$)

Always prefer member functions for Maps and Sets because they understand the internal tree structure.

9. Advanced: `std::for_each`

If you want to perform an action on every element in a range, `for_each` is a powerful alternative to a raw loop.

```
for_each(v.begin(), v.end(), [](int &n) {  
    n *= 2; // Double every number  
});
```

Because it uses iterators, `for_each` is highly readable and works seamlessly with range-based logic.

10. Summary & Best Practices

STL algorithms provide a "vocabulary" for your code. Using `std::sort` instead of writing a bubble sort makes your code instantly understandable to other developers.

- **Optimized:** Hand-written loops rarely beat STL library performance.
- **Safety:** STL algorithms are tested for edge cases (like empty ranges).
- **Sorted Requirement:** Always remember that `binary_search` and `bounds` functions require a sorted container.

Standard Power. Efficient Results.